

Polytechnique Montréal

Département de génie électrique

ELE2200 –Système et Simulation

Devoir 3 (Hiver 2026)

Costa Diego Alfonso
Gagnon Louis

Matricule : 2374165
Matricule : 2295595

Date de remise : 27 février 2026

March 27, 2026

Contents

1	Introduction	2
2	Question 1	2
2.1	a) Modèle d'états non linéaire	2
2.2	b) Modèle linéarisé autour d'un point d'équilibre	3
2.3	c) Linearisation du modèle d'état non linéaire	3
2.4	d) Pôles du système linéarisé	4
2.5	e) Valeurs numériques de la fonction de transfert et des pôles du système linéarisé	5
2.6	f) Négligence du pôle rapide	5
2.7	g) Approximation du système linéarisé par un système d'ordre 2	5
3	Question 2	6
4	Question 3	10
5	Conclusion	10

1 Introduction

Les systèmes à lévitation magnétique sont des systèmes non linéaires instables en boucle ouverte nécessitant une commande pour fonctionner correctement. Dans ce travail, nous modélisons et linéarisons un système de lévitation afin d'en analyser la dynamique. Une fonction de transfert est obtenue pour étudier les pôles et identifier les dynamiques dominantes. Enfin, des simulations permettent de valider le comportement du système en boucle fermée.

2 Question 1

2.1 a) Modèle d'états non linéaire

De l'énoncé les équations suivantes sont données :

$$L \frac{di}{dt} + Ri = u \quad (1)$$

$$m\ddot{h} = F_m - F_g \quad (2)$$

$$F_m = k_f \frac{i^2}{h^2} \quad (3)$$

On demande le modèle d'état du système (u est la tension sur la Fig. 2), sous la forme

$$\dot{\mathbf{x}} = f(\mathbf{x}, u) \quad (4)$$

$$y = g(\mathbf{x}, u), \quad (5)$$

avec f et g des fonctions à déterminer, en prenant $\mathbf{x} = [x_1 \ x_2 \ x_3]^T = [h \ \dot{h} \ i]^T$ et la sortie $y = h$. Le script suivant est exécuté dans Matlab pour trouver les fonctions f et g à partir des équations (1), (2) et (3).

```
1 % Symbolic variables
2 syms m g0 R L kf he real
3
4 % Numerical values
5 m_val = 0.025;           % kg
6 g0_val = 9.81;           % m/s^2
7 R_val = 22.5;            % Ohm
8 L_val = 0.084;           % H
9 kf_val = 2.725e-4;       % N*m^2/A^2
10 he_val = -0.05;         % m
11
12 sym_vars = [m, g0, R, L, kf, he];
13 val_vars = [m_val, g0_val, R_val, L_val, kf_val, he_val];
14
15 syms h dh i u real
16
17 Fm = kf * (i/h)^2;
18 Fg = m*g0;
19
20 ddh = (Fm-Fg)/m;
21 di = (u-R*i)/L;
22
```

```

23 x_phys = [ h dh i];
24 f_phys = [dh ddh di];
25 y_phys = h;
26
27 syms x1 x2 x3 real
28 x_etat = [x1 x2 x3];
29
30 f_etat = subs(f_phys,x_phys,x_etat);
31 g_etat = subs(g_phys,x_phys,x_etat);

```

On obtient

$$\dot{\mathbf{x}} = f(\mathbf{x}, u) = \begin{pmatrix} x_2 \\ -\frac{g_0 m - \frac{k_f x_3^2}{x_1^2}}{m} \\ \frac{u - R x_3}{L} \end{pmatrix} \quad (6)$$

$$y = g(\mathbf{x}, u) = x_1 \quad (7)$$

2.2 b) Modèle linéarisé autour d'un point d'équilibre

Le script suivant est exécuté dans Matlab pour trouver le modèle linéarisé du système autour du point d'équilibre $h_e = -0.05$ m.

```

1 x_1_eq = h_e;
2 eq1 = subs(f_etat(1),x1,x_1_eq)==0;
3 eq2 = subs(f_etat(2),x1,x_1_eq)==0;
4 sol = solve([eq1 eq2], [x2 x3]);

```

On obtient :

$$x_{2,eq} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, x_{3,eq} = \begin{pmatrix} h_e \sqrt{\frac{g_0 m}{k_f}} \\ -h_e \sqrt{\frac{g_0 m}{k_f}} \end{pmatrix} \quad (8)$$

Il y a deux solutions pour $x_{3,eq}$ car l'équation d'équilibre pour le courant i est $i^2 = \frac{mg_0 h_e^2}{k_f}$, ce qui donne deux valeurs possibles : $i = \pm \sqrt{\frac{mg_0 h_e^2}{k_f}}$. La force magnétique dépendant de i^2 , le signe du courant n'affecte pas la force, mais deux équilibres sont possibles. Ici, on choisit la solution $x_{3,eq} = h_e \sqrt{\frac{g_0 m}{k_f}}$.

On obtient :

$$\mathbf{x}_{equilibre} = \begin{pmatrix} h_e & 0 & h_e \sqrt{\frac{g_0 m}{k_f}} \end{pmatrix} = \begin{pmatrix} -0.05 & 0 & -1.5 \end{pmatrix} \quad (9)$$

2.3 c) Linearisation du modèle d'état non linéaire

Le script suivant est exécuté dans Matlab pour trouver le modèle linéarisé.

```

1 % Define the state-space representation
2 A_sym = jacobian(f_etat, x_etat);
3 B_sym = jacobian(f_etat, u);
4 C_sym = jacobian(g_etat, x_etat);
5 D_sym = jacobian(g_etat, u);
6
7 A_val = subs(A_sym, [x_etat sym_vars], [x_equi_val val_vars]);
8 B_val = subs(B_sym, [x_etat sym_vars], [x_equi_val val_vars]);
9 C_val = subs(C_sym, [x_etat sym_vars], [x_equi_val val_vars]);
10 D_val = subs(D_sym, [x_etat sym_vars], [x_equi_val val_vars]);
11
12 A_val_round = vpa(A_val,3);
13 B_val_round = vpa(B_val,3);
14 C_val_round = vpa(C_val,3);
15 D_val_round = vpa(D_val,3);

```

On obtient :

$$A = \begin{bmatrix} 0 & 1 & 0 \\ -\frac{2k_f x_3^2}{m x_1^3} & 0 & \frac{2k_f x_3}{m x_1^2} \\ 0 & 0 & -\frac{R}{L} \end{bmatrix} = \begin{bmatrix} 0 & 1.0 & 0 \\ 392.0 & 0 & -13.1 \\ 0 & 0 & -268.0 \end{bmatrix} \quad (10)$$

$$B = \begin{bmatrix} 0 \\ 0 \\ \frac{1}{L} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 11.9 \end{bmatrix} \quad (11)$$

$$C = [1 \ 0 \ 0] = [1.0 \ 0 \ 0] \quad (12)$$

$$D = 0.0 \quad (13)$$

2.4 d) Pôles du système linéarisé

Le script suivant est exécuté dans Matlab pour trouver les pôles du système linéarisé

```

1 syms s;
2 I = eye(size(A_sym));
3 G_sym = subs(C_sym*inv(s*I - A_sym)*B_sym + D_sym, x_etat, x_equi_sym)
4 [num, den] = numden(G_sym); % separate numerator & denominator
5 poles = solve(den == 0, s)

```

On obtient :

$$G(s) = \frac{2 h e^2 k_f \sqrt{\frac{g_0 m}{k_f}}}{(m h e^3 s^2 + 2 g_0 m h e^2) (R + L s)} \quad (14)$$

$$(15)$$

$$\text{poles} = \begin{bmatrix} -\frac{R}{L} \\ \frac{\sqrt{2}\sqrt{-g_0}}{\sqrt{h e}} \\ -\frac{\sqrt{2}\sqrt{-g_0}}{\sqrt{h e}} \end{bmatrix} \quad (16)$$

2.5 e) Valeurs numériques de la fonction de transfert et des pôles du système linéarisé

Le script suivant est exécuté dans Matlab pour trouver les valeurs numériques de la fonction de transfert et des pôles du système linéarisé

```
1 G_val = collect(subs(G_sym, sym_vars, val_vars));
2 G_val_vpa = vpa(G_val,3)
3 poles_val = subs(poles_sym, sym_vars, val_vars);
4 poles_val_vpa = vpa(poles_val,3)
```

$$G(s) = \frac{7.85e + 20}{-5.04e + 18s^3 - 1.35e + 21s^2 + 1.98e + 21s + 5.3e + 23} \quad (17)$$

$$\text{poles} = \begin{bmatrix} -\frac{R}{L} \\ \frac{\sqrt{2}\sqrt{-g_0}}{\sqrt{h_e}} \\ -\frac{\sqrt{2}\sqrt{-g_0}}{\sqrt{h_e}} \end{bmatrix} = \begin{bmatrix} -\frac{1875}{\sqrt{2}\sqrt{20}\sqrt{981}} \\ \frac{10}{\sqrt{2}\sqrt{20}\sqrt{981}} \\ -\frac{10}{\sqrt{2}\sqrt{20}\sqrt{981}} \end{bmatrix} = \begin{bmatrix} -268.0 \\ 19.8 \\ -19.8 \end{bmatrix} \quad (18)$$

2.6 f) Négligence du pôle rapide

Le pôle $p_1 = -268.0$ est très éloigné des deux autres, ce qui correspond à une dynamique beaucoup plus rapide (constante de temps très faible). Son influence sur la réponse globale du système est donc négligeable à l'échelle de temps des autres pôles.

Ainsi, on peut négliger le pôle rapide $p_1 = -268.0$ et conserver uniquement les pôles dominants $p_2 = 19.8$ et $p_3 = -19.8$ pour l'approximation du système.

2.7 g) Approximation du système linéarisé par un système d'ordre 2

En négligeant le pôle rapide $p_1 = -268.0$, on peut approximer la fonction de transfert du système linéarisé par une fonction de transfert d'ordre 2 qui ne contient que les pôles dominants $p_2 = 19.8$ et $p_3 = -19.8$.

Le script suivant est exécuté dans Matlab pour trouver la fonction de transfert d'ordre 2 approximée

```
1 [num_full, den_full] = numden(G_val);
2 num_full = double(sym2poly(num_full));
3 den_full = double(sym2poly(den_full));
4 K = num_full(end) / den_full(end);
5 poles = double([poles_val(2) poles_val(3)]);
6 den = poly(poles);
7 Ga = tf(K, den/abs(den(end)));
8 [num_G_a, den_G_a] = tfdata(Ga, 'v');
9 Ga_sym = poly2sym(num_G_a, s) / poly2sym(den_G_a, s);
10 Ga_sym_vpa = vpa(Ga_sym,3);
```

On obtient la fonction de transfert d'ordre 2 approximée :

$$G_a(s) = \frac{1}{675 \left(\frac{5s^2}{1962} - 1 \right)} = \frac{0.00148}{0.00255s^2 - 1.0} \quad (19)$$

3 Question 2

Pour trouver la fonction de transfert $\frac{\Delta H(s)}{\Delta H_c(s)}$ en fonction de K_p et K_d , on peut faire de l'algèbre des diagrammes. D'abord, on part du diagramme fonctionnel de la figure 3 de l'énoncé du devoir:

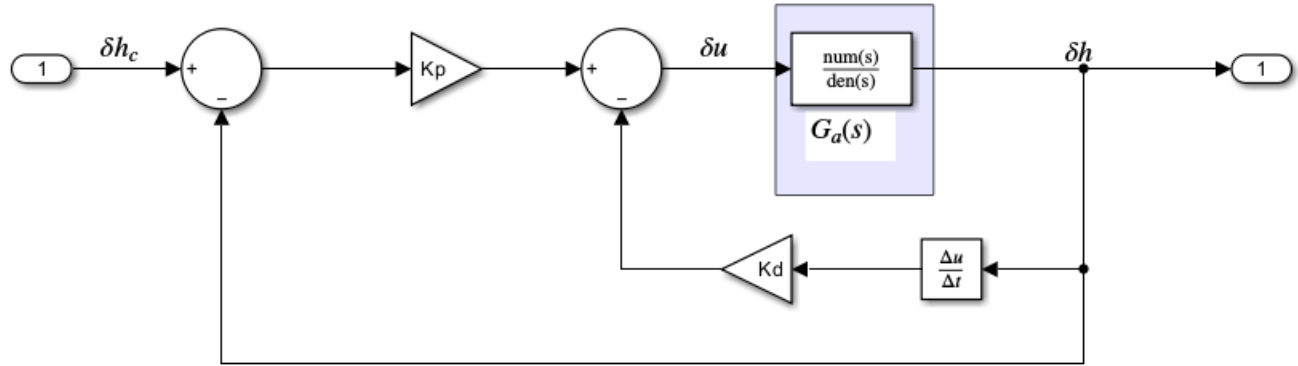


Figure 1: Diagramme fonctionnel initial

On commence par combiner les blocs en cascade:

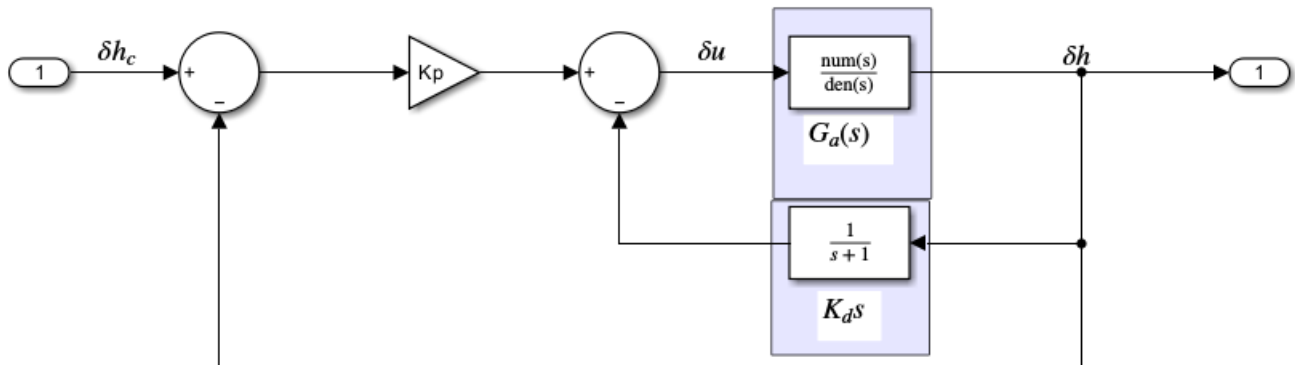


Figure 2: Étape 1

Ensuite, on réduit la boucle de rétroaction négative en un seul bloc:

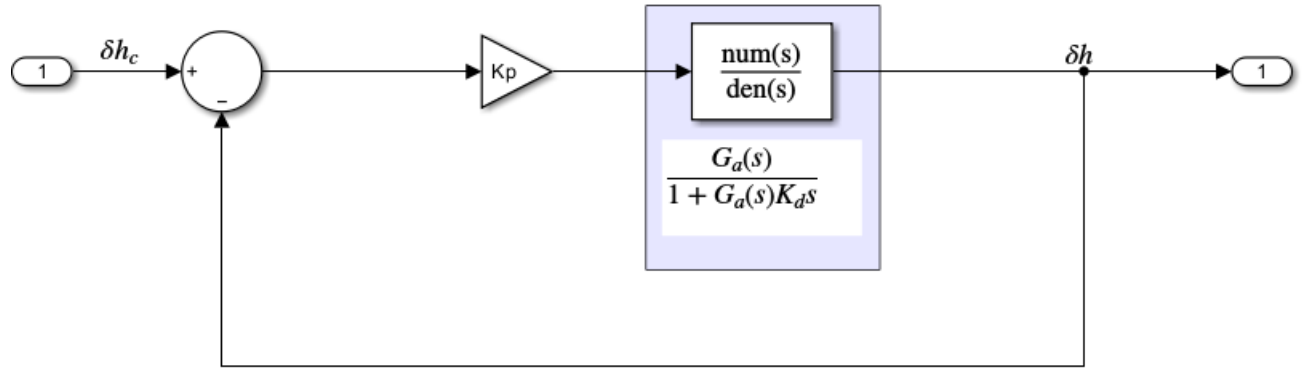


Figure 3: Étape 2

Ensuite, en combinant les blocs en cascade, on obtient:

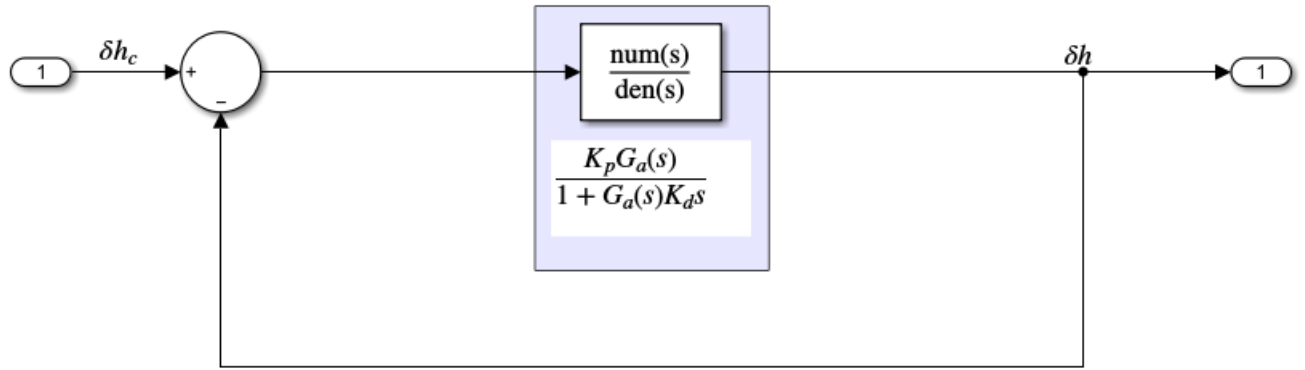


Figure 4: Étape 3

Le diagramme fonctionnel final ci-dessus est une simple boucle de rétroaction négative dont le bloc de transfert est donné par:

$$\frac{\frac{K_p G_a(s)}{1 + G_a(s)K_d s}}{1 + \frac{K_p G_a(s)}{1 + G_a(s)K_d s}} \Rightarrow \frac{\Delta H(s)}{\Delta H_c(s)} = \frac{K_p G_a(s)}{1 + G_a(s)K_d s + K_p G_a(s)}$$

Puisqu'il a été établi dans la question 1, que la forme numérique de $G_a(s)$ est

$$G_a(s) = \frac{0.58}{s^2 - 392.4}$$

alors

$$\frac{\Delta H(s)}{\Delta H_c(s)} = \frac{K_p \left(\frac{0.58}{s^2 - 392.4} \right)}{1 + \left(\frac{0.58}{s^2 - 392.4} \right) K_d s + \left(\frac{0.58}{s^2 - 392.4} \right) K_p} = \frac{0.58 K_p}{s^2 - 392.4 + 0.58 K_d s + 0.58 K_p}$$

et donc

$$\boxed{\frac{\Delta H(s)}{\Delta H_c(s)} = \frac{0.58 K_p}{s^2 + 0.58 K_d s - 392.4 + 0.58 K_p}}$$

b) Identification à l'ordre 2

On cherche à déterminer le modèle d'ordre 2 correspondant aux données expérimentales du fichier "data_Devoir_3.mat". On cherche à faire une identification à l'ordre 2, donc à trouver les paramètres K_s (gain statique), ξ (coefficient d'amortissement) et ω_n dans le modèle général

$$\frac{\Delta H(s)}{\Delta H_c(s)} = \frac{K_s \omega_n^2}{s^2 + 2\xi \omega_n s + \omega_n^2}$$

On commence par déterminer le gain statique avec l'équation $\frac{\delta h_{RP}}{\delta h_c}$. Il est mentionné dans l'énoncé que l'entrée du système dans les données expérimentales est un échelon de $\delta h_c = 0.01\text{m}$. Pour trouver la valeur de la sortie en régime permanent, nous avons essentiellement trouvé la valeur du dernier élément du vecteur "y" (donné dans le fichier "data_Devoir_3.mat") avec le script matlab suivant:

```
1 load("data_Devoir_3.mat");
2 display(y(end))
```

En faisant exécuter le code ci-dessus, on obtient que $\delta h_{RP} = 1.37\text{m}$. Donc,

$$\Rightarrow K_s = \frac{\delta h_{RP}}{\delta h_c} = \frac{0.0137\text{m}}{0.01\text{m}} = 1.37$$

Ensuite, on calcule le dépassement avec l'équation

$$D = \frac{\delta h_{max} - \delta h_{RP}}{\delta h_{RP}}$$

Pour δh_{max} , nous avons trouvé la valeur maximale du vecteur "y" dans les données expérimentales avec le script matlab suivant:

```
1 load("data_Devoir_3.mat");
2 display(max(y))
```

En faisant exécuter le code ci-dessus, nous avons obtenu la valeur $\delta h_{max} = 0.0143\text{m}$.

Donc,

$$D = \frac{\delta h_{max} - \delta h_{RP}}{\delta h_{RP}} = \frac{0.0143m - 0.0137m}{0.0137m} = 0.04379562$$

Avec le dépassement, on peut calculer le coefficient d'amortissement selon l'équation suivante:

$$D = e^{-\frac{\pi\xi}{\sqrt{1-\xi^2}}} \iff \xi = \sqrt{\frac{(\ln(D))^2}{\pi^2 + (\ln(D))^2}} = \sqrt{\frac{(\ln(0.04379562))^2}{\pi^2 + (\ln(0.04379562))^2}}$$

$$\Rightarrow \xi = 0.70559718$$

Ensuite, on trouve la pulsation naturelle ω_n avec l'équation suivante:

$$T_p = \frac{\pi}{\omega_n \sqrt{1-\xi^2}} \iff \omega_n = \frac{\pi}{T_p \sqrt{1-\xi^2}}$$

Pour trouver le temps de pic, nous avons trouvé le temps correspondant à la valeur maximale du vecteur "y" des données expérimentales avec le script matlab suivant:

```
1 load("data_Devoir_3.mat");
2 [ligne, colonne] = find(y == max(y));
3 display(ligne)
```

En exécutant le script ci-dessus, on obtient la valeur $T_p = 0.1389s$. Donc,

$$\Rightarrow \omega_n = \frac{\pi}{0.1389 \sqrt{1-0.70559718^2}} \Rightarrow \omega_n = 31.91820131 \frac{\text{rad}}{s}$$

Donc,

$$\frac{\Delta H(s)}{\Delta H_c(s)} = \frac{(1.37)(31.91820131)^2}{s^2 + 2(0.70559718)(31.91820131)s + (31.91820131)^2}$$

$$\Rightarrow \frac{\Delta H(s)}{\Delta H_c(s)} = \frac{1395.717058}{s^2 + 45.04278567s + 1018.771575} \quad (\text{FT de l'identification})$$

Nous avons ensuite écrit le script suivant sur matlab:

```
1 load("data_Devoir_3.mat");
2 s = tf('s');
3 deltaH_identification = (1395.717058)/(s^2 + 45.04278567*s + 1018.771575);
4 u = ones(127,1)*0.01;
5 [y_identification, t_identification] = lsim(deltaH_identification,u,t);
6 plot(t,y, 'b')
7 hold on
8 plot(t_identification, y_identification, 'r');
9 title("R\`eponses $\delta h$ a un \`echelon $\delta h_{c}=0.01m$", 'Interpreter','latex');
10 grid on
11 xlabel('t (en secondes)', 'FontSize',9)
12 ylabel('$\delta h$ (en metres)', 'Interpreter','latex', 'FontSize',9)
13 legend("exp\`erimentale", 'identification', 'Location', 'southeast')
14 print -dpng Demarches_devoir3.png
```

En exécutant le script matlab ci-dessus, on obtient le graphe suivant:

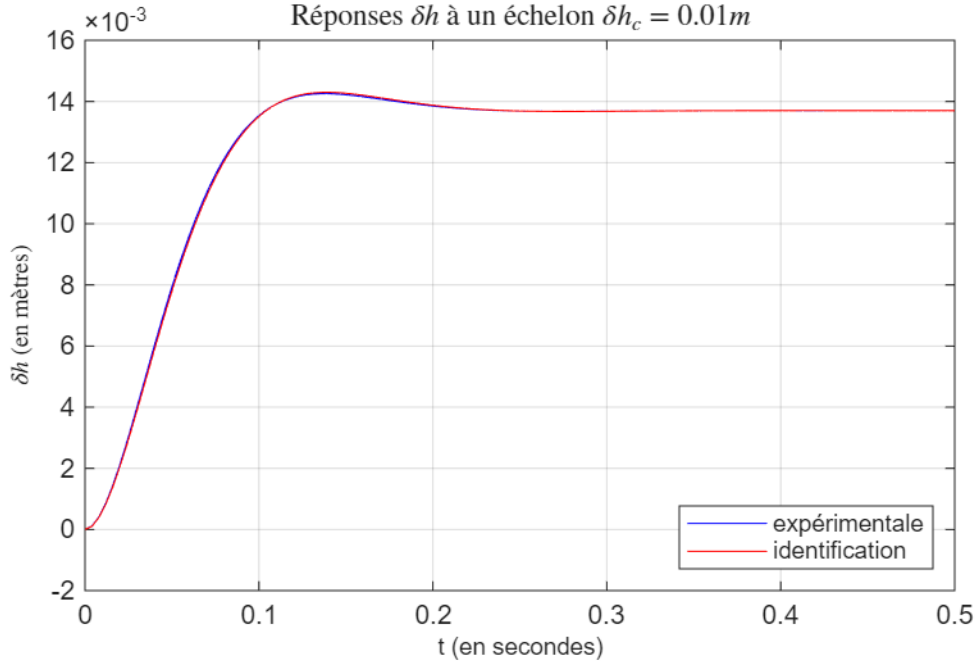


Figure 5: Réponse expérimentale et réponse provenant de l'identification

On peut voir dans la figure ci-dessus que la réponse expérimentale est presque identique à la réponse provenant de l'identification qu nous avons effectué plus tôt dans ce rapport. Cela nous indique que la fonction de transfert trouvée lors de notre identification représente très bien le système en boucle fermée linéarisé. On peut donc déduire la valeur des gains K_p et K_d en comparant la forme de $\frac{\Delta H(s)}{\Delta H_c(s)}$ trouvée en a) à celle découlant de l'identification à l'ordre 2:

$$\frac{0.58K_p}{s^2 + 0.58K_d s - 392.4 + 0.58K_p} = \frac{1395.717058}{s^2 + 45.04278567s + 1018.771575}$$

On fait tendre s vers 0 pour comparer les gains statiques:

$$\frac{0.58K_p}{-392.4 + 0.58K_p} = \frac{1395.717058}{1018.771575} \iff 0.58K_p = -537.5880001 + 0.7946K_p \iff \boxed{K_p = 2505.07}$$

Ensuite, par comparaison des coefficients multipliant "s" au dénominateur:

$$0.58K_d = 45.04278567 \iff \boxed{K_d = 77.66}$$

4 Question 3

a) Diagramme fonctionnel du modèle non-linéaire

Le diagramme fonctionnel que nous avons fait sous simulink est le suivant:

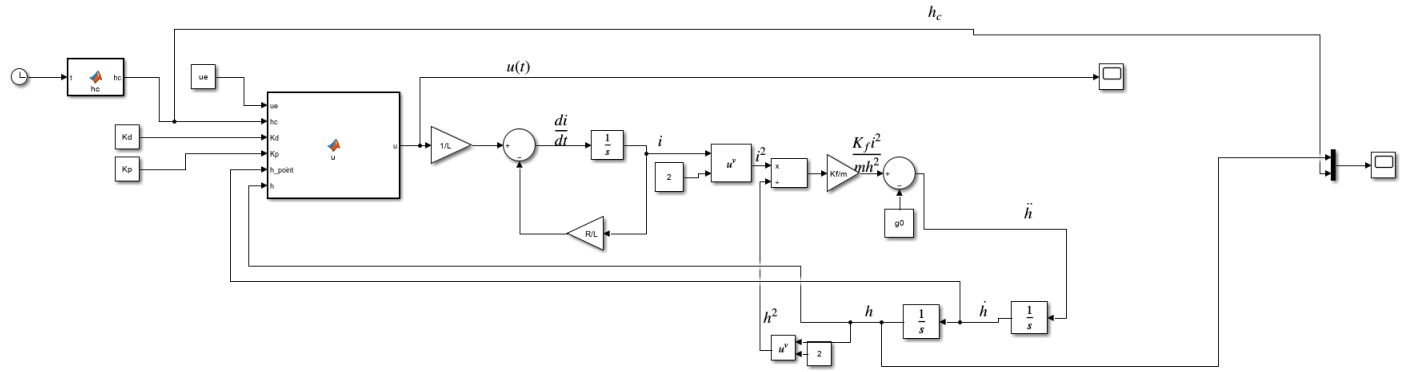


Figure 6: Diagramme fonctionnel du modèle non-linéaire

Le bloc de fonction matlab vers le centre gauche du diagramme représente la loi de commande suivante:

$$u = u_e - K_d \dot{h} - K_p (h - h_c)$$

Le corps de la fonction (dans le bloc "matlab function") est:

```
1 function u = u( ue, hc, Kd, Kp, h_point, h)
2 u = ue - Kd*h_point - Kp*(h - hc);
3 end
```

De plus, nous avons inclus les conditions initiales d'équilibre dans les intégrateurs, i.e. que nous avons inscrit $i_e = 1.5A$ dans l'intégrateur qui a $\frac{di}{dt}$ en entrée et i en sortie, $\dot{h}_e = 0m$ dans l'intégrateur qui a $\ddot{h}$ en entrée et $\dot{h}$ en sortie ainsi que $h_e = -0.05m$ dans l'intégrateur qui a $\dot{h}$ en entrée et h en sortie.

b) Réponses temporelles

D'abord, nous avons implémenté la fonction h_c définie par morceaux dans l'énoncé avec la fonction suivante (contenu du bloc "matlab function" de gauche du diagramme simulink en a)):

```
1 function hc = hc(t)
2 if (0 <= t) && (t < 5)
3     hc = -0.05;
4 elseif (5 <= t) && (t < 10)
5     hc = -0.075;
6 elseif (10 <= t) && (t < 15)
7     hc = -0.06;
8 elseif (15 <= t) && (t < 20)
9     hc = -0.055;
10 else
11     hc = -0.05;
12 end
```

Ensuite, pour tracer les graphes de $h(t)$ et $h_c(t)$ superposés ainsi que $u(t)$ séparément, nous avons écrit le script matlab suivant:

```

1 %Question 3b)
2 %Valeurs numérique des variables
3 m = 0.025;
4 g0 = 9.81;
5 R = 22.5;
6 L = 0.084;
7 Kf = 2.725*10^-4;
8 he = -0.05;
9 Kp = 3050;
10 Kd = 105;
11 ue = 33.75;
12 ie = 1.5;
13 h_prime_e = 0;
14 %Tracer les graphes de h(t) et de hc(t)
15 out_Q3 = sim('SchemaIntegrateurQ3a.slx','Solver','ode45','FixedStep','
    0.001','StopTime','35')
16 plot(out_Q3.h_hc_t(:,1),out_Q3.h_hc_t(:,2),'Color','blue')
17 hold on
18 plot(out_Q3.h_hc_t(:,1),out_Q3.h_hc_t(:,3),'Color','red')
19 grid on
20 title("Graphe de $h(t)$ et de $h_{c}(t)$",'Interpreter','latex')
21 legend('$h(t)$','$h_{c}(t)$','Interpreter','latex')
22 xlabel('t (secondes)','Interpreter','latex','FontSize',9)
23 ylabel('$h(t)$ et $h_{c}(t)$ (m\`etres)','Interpreter','latex','FontSize',
    9)
24 hold off
25 print -dpng graphe_h_hc_t.png
26 %Tracer le graphe de u(t)
27 plot(out_Q3.u_t(:,1),out_Q3.u_t(:,2),'Color','magenta')
28 grid on
29 title("Graphe de la tension d'entr\'ee u(t)","Interpreter","latex")
30 xlabel('t (secondes)', 'Interpreter','latex')
31 ylabel('u (volts)','Interpreter','latex')
32 print -dpng graphe_u_t.png

```

En exécutant le script matlab ci-dessus, on obtient les graphes suivants:

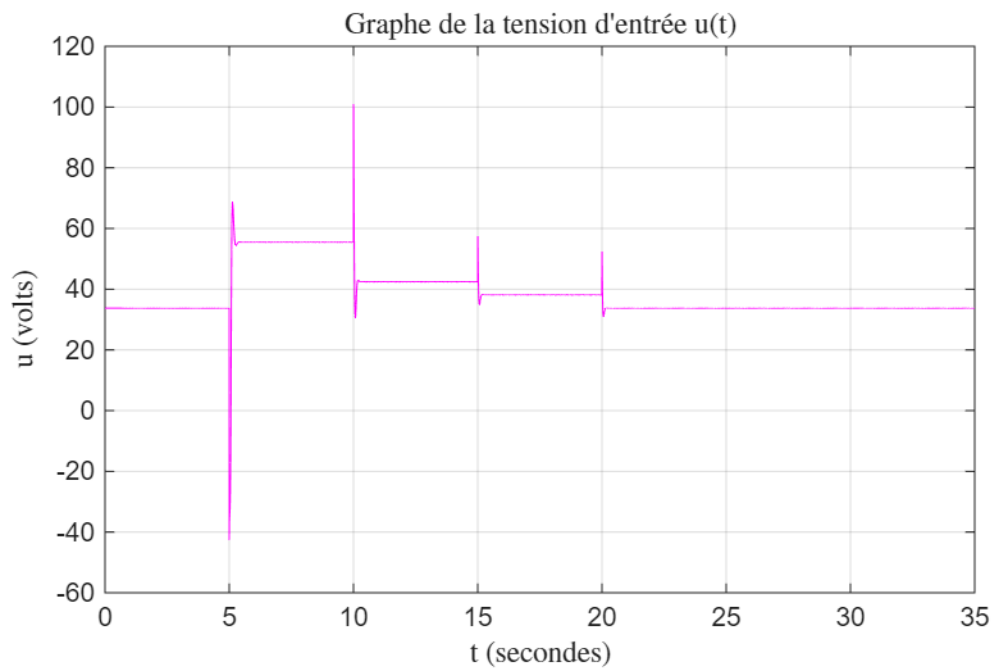


Figure 7: Graphe de $u(t)$

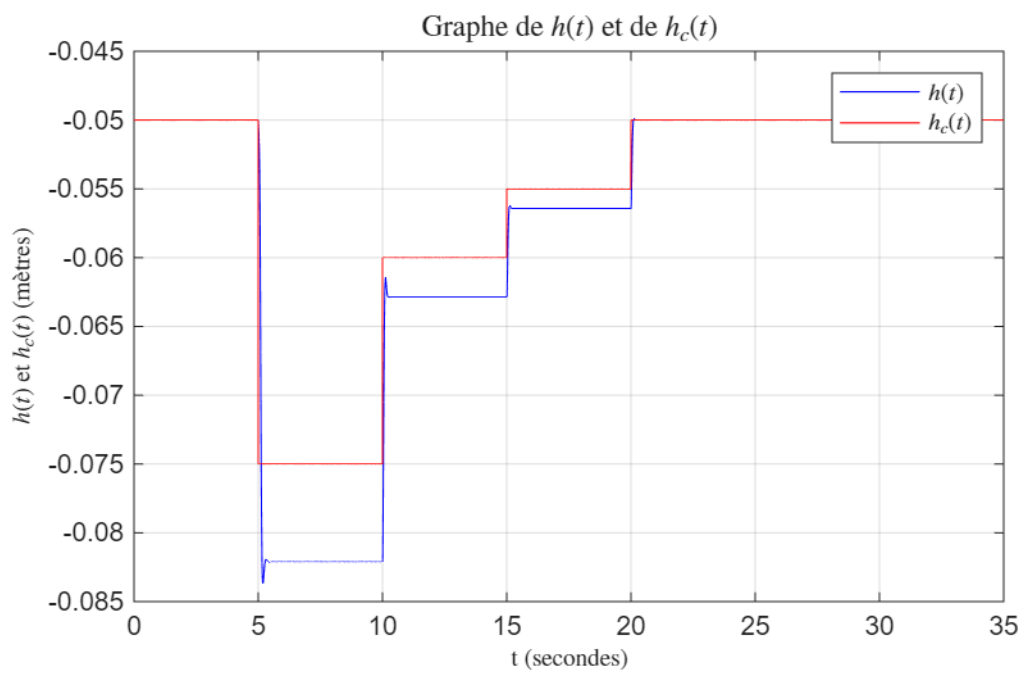


Figure 8: Graphe de $h(t)$ et de $h_c(t)$

En ce qui concerne le graphe de $h(t)$ et de $h_c(t)$ On peut voir dans les graphes ci-dessus que la hauteur $h(t)$ suit la tendance "en escalier" de la consigne $h_c(t)$ mais également qu'il y a un écart non-négligeable en ordonnée (hauteur) entre les deux courbes à chaque sous-intervalle de définition de $h_c(t)$. On peut aussi remarquer qu'il n'y a presque pas de décalage (en t) entre les deux courbes, i.e. que lorsque la consigne $h_c(t)$ change de manière subite la hauteur $h(t)$ suit presque immédiatement le changement (mais h n'atteint jamais exactement la même valeur que la consigne h_c , comme mentionné précédemment). On remarque l'apparition de petits pics aux points de discontinuité (ou de "cassure") de la fonction $h_c(t)$. On observe également que les écarts entre h et h_c sont décroissants, i.e. que l'écart entre les deux courbes dans l'intervalle $t \in [15, 20]$ s est plus petit que celui dans l'intervalle $t \in [10, 15]$, qui est lui-même plus petit que l'écart dans l'intervalle $t \in [5, 10]$

En ce qui concerne le graphe de la tension d'entrée $u(t)$, on observe que la courbe suit une tendance "en escalier" qui est due à $h_c(t)$ qui est une fonction définie par morceaux. On observe également la présence de grands "pics" de tension aux points de discontinuité de la fonction $h_c(t)$, soit aux instants correspondants à 5, 10, 15 et 20 secondes. On observe aussi que les pics ne sont pas tous de même taille.

c) Modification du correcteur

Comme mentionné en b), on observe dans le graphe des courbes $h(t)$ et $h_c(t)$ que la hauteur $h(t)$ ne suit pas exactement le comportement de la consigne $h_c(t)$. Pour réduire l'écart entre $h(t)$ et $h_c(t)$, il faudrait modifier la loi de commande pour qu'elle puisse réduire l'écart entre $h(t)$ et $h_c(t)$. On pourrait par exemple additionner un intégrateur à la loi de commande (correcteur) actuelle, qui aurait l'entrée $(h - h_c)$. Cela permettrait, pour chaque sous-intervalle dans la définition de $h_c(t)$, de faire tendre l'erreur (entre la consigne et la sortie) vers 0. Pour être plus précis, la nouvelle loi de commande aurait la forme suivante (ou C serait une constante positive):

$$u = u_e - K_d \dot{h} - K_p(h - h_c) + C \int (h - h_c) dt$$

Si $(h - h_c) > 0$, alors le terme $-K_p(h - h_c)$ serait négatif et serait graduellement annulé par l'intégrale (qui serait positive et croissante) lorsque t augmente (car l'intégrale d'une constante donne une constante multipliée par t). D'autre part, si $(h - h_c) < 0$, alors le terme $-K_p(h - h_c)$ serait positif et donc l'intégrale serait négative et décroissante et annulerait graduellement $-K_p(h - h_c)$. Dans les deux cas, u tendrait vers u_e (la tension d'équilibre) et l'écart entre h et h_c tendrait vers 0 (avec suffisamment de temps).

5 Conclusion

conclu text